Midterm Project Group 8

Group members: Ryan Merlino, Nikolas Platon, April Walter

"How is the number of diagnoses related to hospital readmission, and does that relationship differ by admission type (Emergency, Urgent, Elective)?"

In [ ]:
```
#Midterm Project Group 8 - Ryan Merlino, Nikolas Platon, April Walter
#https://github.com/apriljwalter-lgtm/Group_8_Midterm_project
```

In [ ]:
```
# Import the libraries we need. marimo builds the notebook and shows the
text, pandas loads and works with the data table, and matplotlib and
seaborn draw the charts.
import marimo as mo
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [ ]:
```
# step 1 - Load the data from GitHub so the notebook also runs online at
marimo.app. We look at the first rows to make sure the data loaded
correctly and to see what the columns look like before we start.
df = pd.read_csv("https://raw.githubusercontent.com/apriljwalter-
lgtm/Group_8_Midterm_project/main/diabetes_130.csv")
df.head()
```

	age	gender	admission_type	time_in_hospital	num_lab_procedures	num_procedur
0	70-79	Male	Elective	4	27	
1	80+	Female	Elective	11	70	
2	60-69	Male	Emergency	5	69	
3	70-79	Female	Emergency	5	73	
4	60-69	Male	Urgent	5	37	

In [ ]:
```
# Step 2- Check size, column types, and missing values. We do this before
the analysis so we know how many patients we have and if any data needs to
be cleaned first.
print(df.shape)
print(df.dtypes)
print("Missing values:", df.isna().sum().sum())
```

```
(3000, 10)
age                     str
gender                  str
admission_type          str
time_in_hospital      int64
num_lab_procedures    int64
num_procedures        int64
num_medications       int64
number_diagnoses      int64
diabetes_med            str
readmitted              str
dtype: object
Missing values: 0
```

In [ ]:
```
# Step 3 Make a new data frame = data with only the columns our question
uses. Only columns are removed, there are no patients removed.

# Step 4 We add a new column that turns the readmission text into a number:
0 = not readmitted, 1 = readmitted so the numbers are easier to work with.

#Step 5 We sort each patient into one of four diagnosis ranges so we can
compare groups instead of 12 separate values. The function checks the
number with if/elif/else and returns a label. .apply() runs that function
on every row.

#These steps need to group together so we can call the values later in the
charting.
```

In [ ]:
```
#step 3 make a new data frame = data
data = df[["admission_type", "number_diagnoses", "readmitted"]].copy()

# Step 4 Computed column: 1 = readmitted (any time), 0 = not readmitted.
data["readmitted_flag"] = data["readmitted"].map({
    "Not readmitted": 0,
    "After 30 days": 1,
    "Within 30 days": 1,
})

# Step 5 – Computed column: put the number of diagnoses into four ranges.
def diagnosis_group(n):
    if n <= 4:
        return "1–4"
    elif n <= 6:
        return "5–6"
    elif n <= 8:
        return "7–8"
    else:
        return "9+"

data["diagnosis_group"] = data["number_diagnoses"].apply(diagnosis_group)
data.head()
```

	admission_type	number_diagnoses	readmitted	readmitted_flag	diagnosis_group
0	Elective	5	Not readmitted	0	5-6
1	Elective	9	Within 30 days	1	9+
2	Emergency	9	Within 30 days	1	9+
3	Emergency	9	Not readmitted	0	9+
4	Urgent	5	Not readmitted	0	5-6

In [ ]:
```
# Step 6- Keep only the rows where admission_type is "Emergency" and save
them as a new Dataframe, emergency. This filter is only for this step since
its the largest group so its numbers are the most reliable. We use it to
check if readmission still rises with more diagnoses within a single
admission type. How the filter works: data["admission_type"] == "Emergency"
checks every row and gives True or False. Putting that inside data[ ] keeps
only the True rows.
```

In [ ]:
```
# Filter: keep only Emergency admissions.
# This smaller table is only used for this check.
emergency = data[data["admission_type"] == "Emergency"]

print("All patients:      ", len(data))
print("Emergency patients:", len(emergency))

# Readmission rate (%) by diagnosis group, Emergency patients only.
emergency_rates = emergency.groupby("diagnosis_group")
["readmitted_flag"].mean() * 100
emergency_rates.round(1)
```

```
All patients:       3000
Emergency patients: 1716
```

diagnosis_group	readmitted_flag
1-4	24.4
5-6	34.6
7-8	47.3
9+	46.3

In [ ]:
```
# Step 7- Calculate the readmission rate percentage for each admission
type. We look at admission type by itself first so we can see if Emergency,
Urgent and Elective patients get readmitted at different rates before we
add in the number of diagnoses.
```

In [ ]:
```
# Readmission rate (%) for each admission type.
by_admission = data.groupby("admission_type")["readmitted_flag"].mean() *
100
by_admission.round(1)
```

admission_type	readmitted_flag
Elective	34.1
Emergency	41.8
Urgent	41.4

In [ ]:
```
# Step 8- Calculate the readmission rate by admission type and diagnosis
group. We need both together to answer the second part of our question, if
the relationship between diagnoses and readmission is different for each
admission type. This table is also used for the line chart later.
```

In [ ]:
```
# Readmission rate (%) for each admission type + diagnosis group.
by_type_and_dx = data.groupby(["admission_type", "diagnosis_group"])
["readmitted_flag"].mean() * 100
by_type_and_dx =
by_type_and_dx.round(1).reset_index(name="readmission_rate")
by_type_and_dx
```

	admission_type	diagnosis_group	readmission_rate
0	Elective	1-4	21.0
1	Elective	5-6	27.0
2	Elective	7-8	36.6
3	Elective	9+	42.2
4	Emergency	1-4	24.4
...	...	...	...
7	Emergency	9+	46.3
8	Urgent	1-4	29.7
9	Urgent	5-6	30.7
10	Urgent	7-8	48.0
11	Urgent	9+	48.4

12 rows × 3 columns

In [ ]:
```
#Step 9 – Sort the data so highest readmission rates are first.
```

In [ ]:
```
# Highest readmission rate first.
sorted_groups = by_type_and_dx.sort_values("readmission_rate",
```

```
ascending=False)
sorted_groups
```

	admission_type	diagnosis_group	readmission_rate
11	Urgent	9+	48.4
10	Urgent	7-8	48.0
6	Emergency	7-8	47.3
7	Emergency	9+	46.3
3	Elective	9+	42.2
...	...	...	...
9	Urgent	5-6	30.7
8	Urgent	1-4	29.7
1	Elective	5-6	27.0
4	Emergency	1-4	24.4
0	Elective	1-4	21.0

12 rows × 3 columns

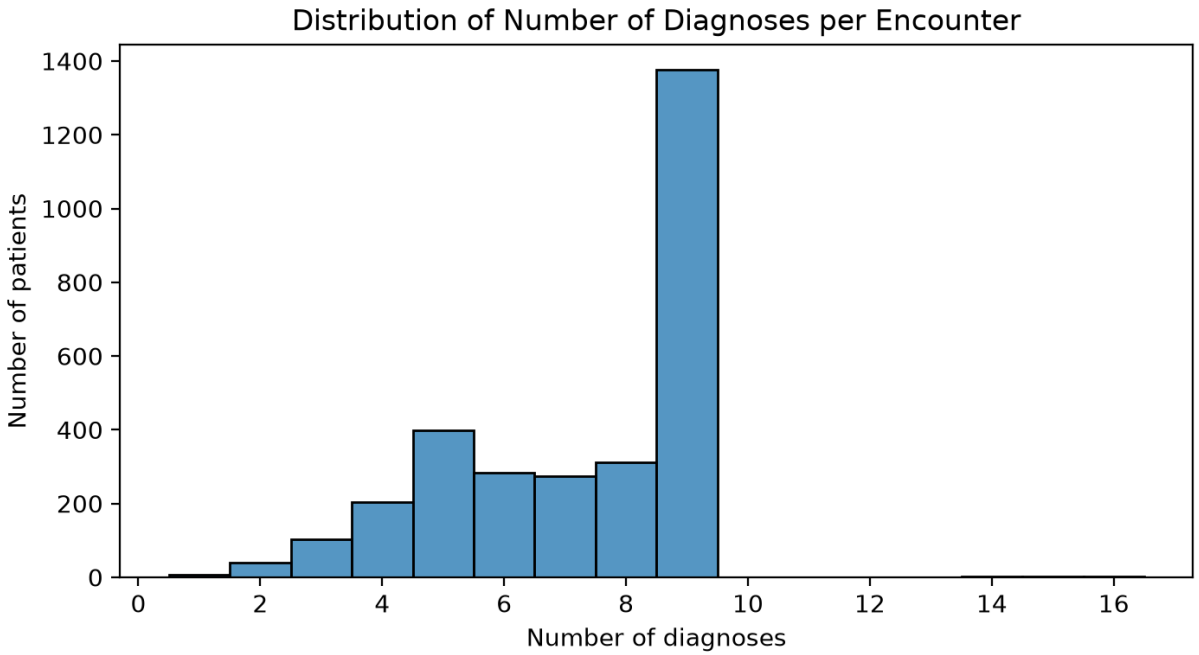
In [ ]:
```
#Step 10- Create a chart settings cell so the charts list the categories in the same order each time. Without this the charts would sort the categories on their own, so Elective, Urgent and Emergency could show up in a different order on each chart and make them harder to compare.
```

In [ ]:
```
readmit_order = ["Not readmitted", "After 30 days", "Within 30 days"]
admission_order = ["Elective", "Urgent", "Emergency"]
```

Histogram- Distribution of Number of Diagnoses per Encounter

We use a histogram because it shows how many patients have each number of diagnoses. This helps us see which numbers are common and which are rare before we compare readmission. It is also why we put the diagnoses into groups and leave out the rare values in the scatter plot.

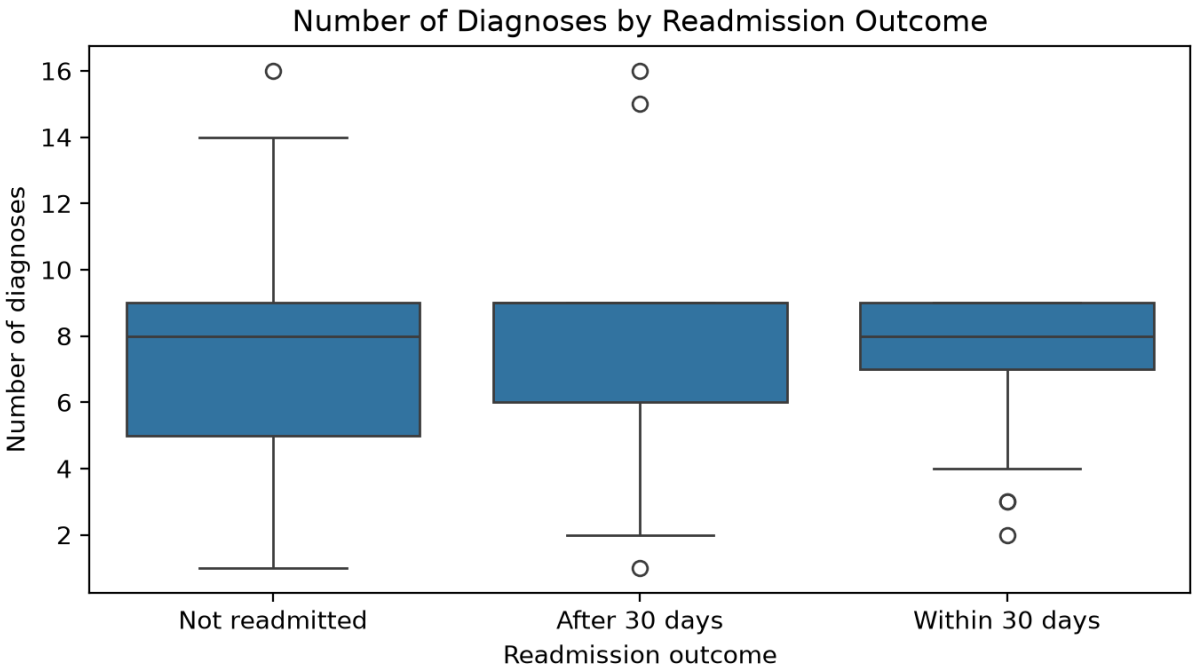
In [ ]:
```
plt.figure(figsize=(8, 4))
sns.histplot(data=data, x="number_diagnoses", discrete=True)
plt.title("Distribution of Number of Diagnoses per Encounter")
plt.xlabel("Number of diagnoses")
plt.ylabel("Number of patients")
plt.gca()
```



Boxplot- Number of Diagnoses by Readmission Outcome

We use a boxplot because it lets us compare the number of diagnoses across the three readmission outcomes. We are looking to see if patients who were readmitted tend to have more diagnoses than patients who were not readmitted.

In [ ]:
```
plt.figure(figsize=(8, 4))
sns.boxplot(data=data, x="readmitted", y="number_diagnoses",
order=readmit_order)
plt.title("Number of Diagnoses by Readmission Outcome")
plt.xlabel("Readmission outcome")
plt.ylabel("Number of diagnoses")
plt.gca()
```

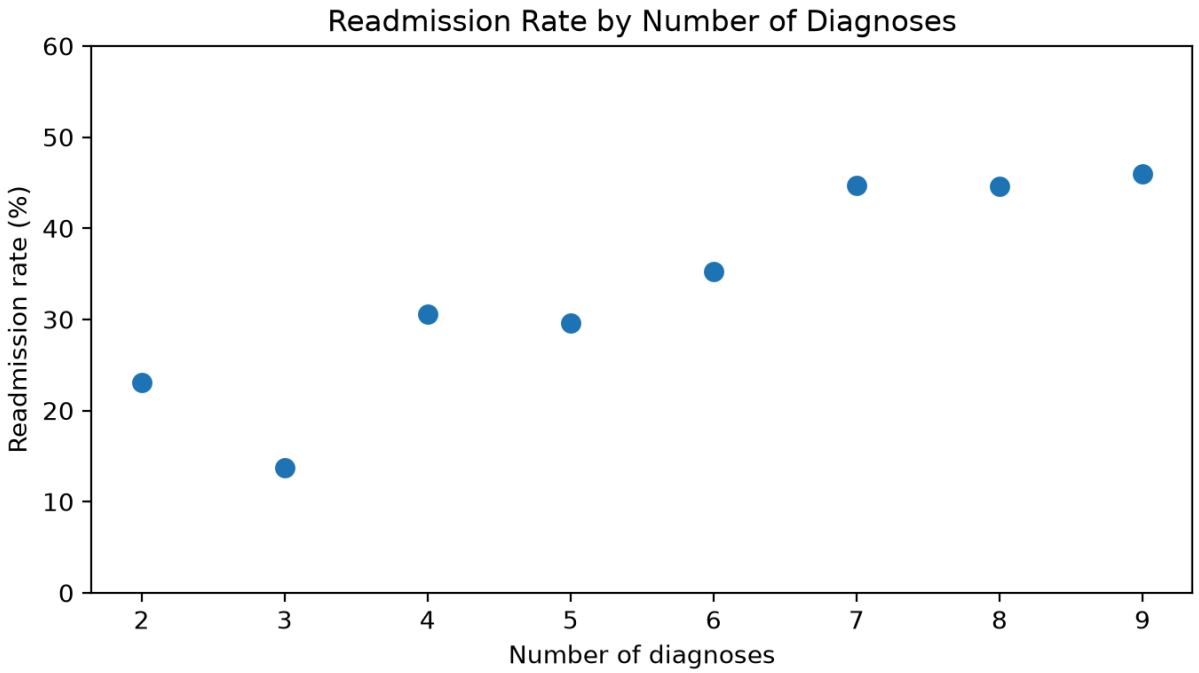


Scatterplot- Readmission Rate by Number of Diagnoses

We use a scatter plot because it shows how two numbers move together, here the number of diagnoses and the readmission rate. We are looking to see if the readmission rate goes up as the number of diagnoses goes up. We only show 2 to 9 diagnoses because the other values have too few patients (1 diagnosis has 6 patients and 14 to 16 have only 1 or 2 each), so their rates would not be reliable. Even within 2 to 9, the points at 2 and 3 diagnoses rest on small groups (39 and 102 patients), so the low end is noisy; the overall upward trend is clearer from 4 diagnoses onward.

In [ ]:
```
# Readmission rate (%) for each number of diagnoses.
rate_by_dx = data.groupby("number_diagnoses")["readmitted_flag"].mean() *
100
rate_by_dx = rate_by_dx.reset_index(name="readmission_rate")
# Keep only 2 to 9 diagnoses, the other values have too few patients to
give a reliable rate.
rate_by_dx = rate_by_dx[rate_by_dx["number_diagnoses"].between(2, 9)]

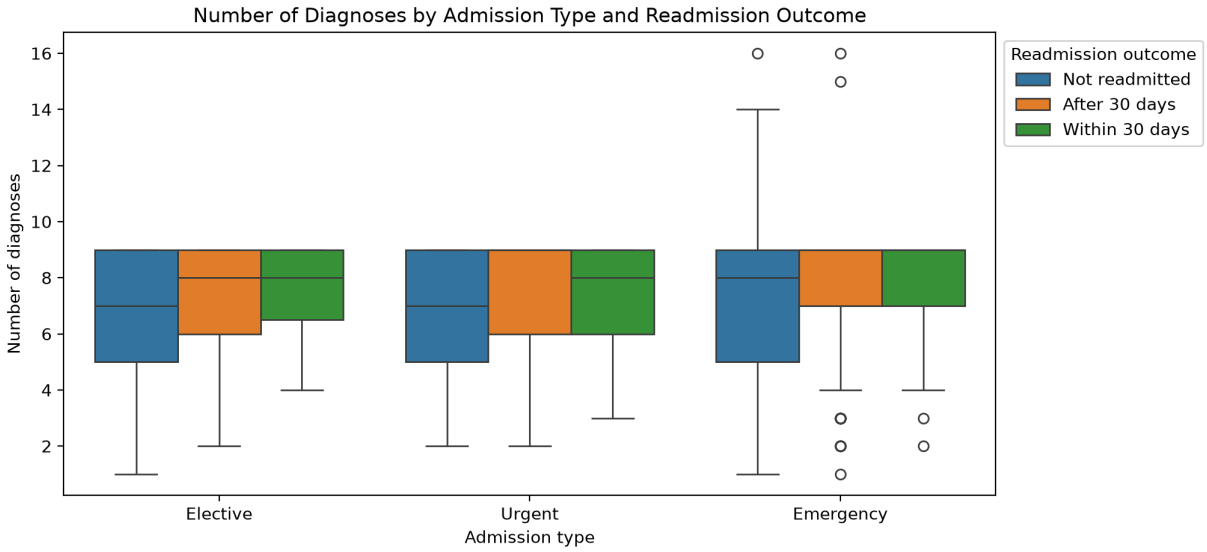
plt.figure(figsize=(8, 4))
sns.scatterplot(data=rate_by_dx, x="number_diagnoses",
y="readmission_rate", s=80)
plt.title("Readmission Rate by Number of Diagnoses")
plt.xlabel("Number of diagnoses")
plt.ylabel("Readmission rate (%)")
plt.ylim(0, 60)
plt.gca()
```



Group Comparison- Number of Diagnoses by Admission Type and Readmission Outcome

We use a grouped boxplot so we can compare readmission outcomes within each admission type side by side. We are looking to see if the gap in diagnoses between readmitted and not readmitted patients is the same for Emergency, Urgent and Elective.

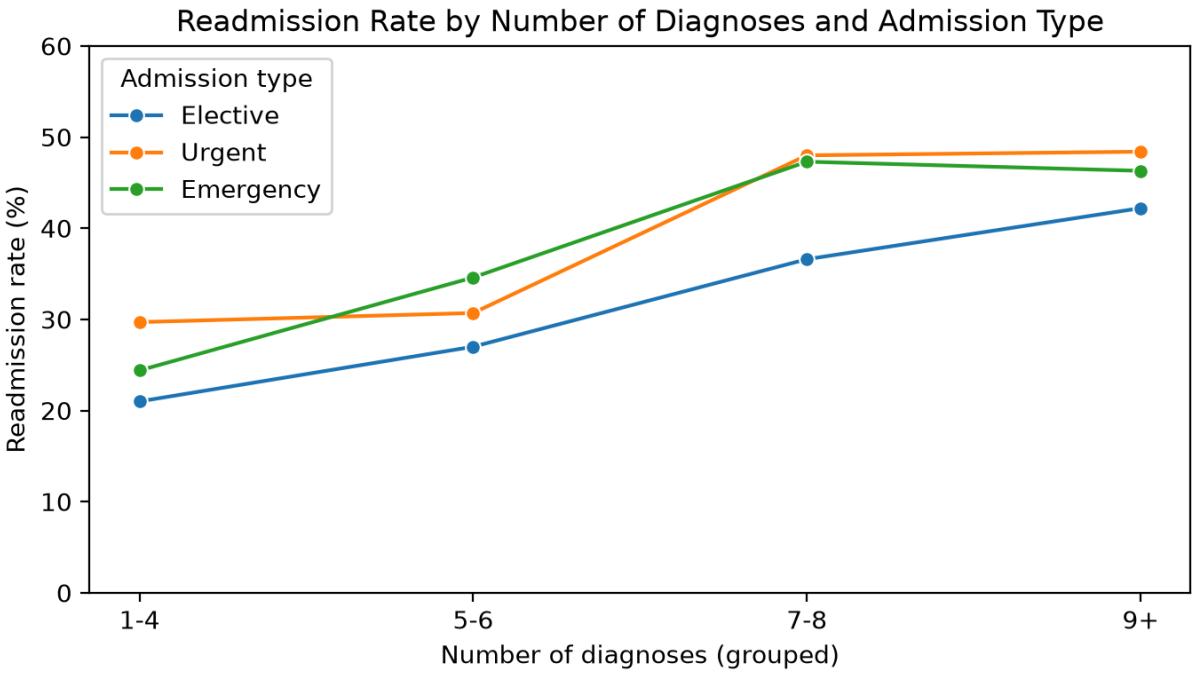
In [ ]:
```
plt.figure(figsize=(10, 5))
sns.boxplot(data=data, x="admission_type", y="number_diagnoses",
hue="readmitted",
            order=admission_order, hue_order=readmit_order)
plt.title("Number of Diagnoses by Admission Type and Readmission Outcome")
plt.xlabel("Admission type")
plt.ylabel("Number of diagnoses")
plt.legend(title="Readmission outcome", loc="upper left", bbox_to_anchor=
(1, 1))
plt.gca()
```



Line Chart- Readmission Rate by Number of Diagnoses and Admission Type

We use a line chart because the diagnosis groups are in order from low to high, so a line shows the trend. Each admission type gets its own line, which lets us see if readmission rises the same way for all three types or if one is different. This chart answers the second part of our research question.

In [ ]:
```
plt.figure(figsize=(8, 4))
sns.lineplot(data=by_type_and_dx, x="diagnosis_group",
y="readmission_rate",
             hue="admission_type", hue_order=admission_order, marker="o")
plt.title("Readmission Rate by Number of Diagnoses and Admission Type")
plt.xlabel("Number of diagnoses (grouped)")
plt.ylabel("Readmission rate (%)")
plt.ylim(0, 60)
plt.legend(title="Admission type")
plt.gca()
```



Throughout our findings, we noticed that 40% of all encounters were followed by a readmission at any time. Readmission rate also rises from 24.6% for patients that have 1-4 diagnoses to 31.9% for those that have 5-6 diagnoses, 44.6% to those with 7-8 diagnoses, and 46% with 9+ diagnoses to which it levels off after approximately 7 diagnoses. Elective admissions also had a lower readmission rate (34.1%) than Urgent (41.4%) and Emergency (41.8%), and Elective is the lowest line in every diagnosis group. Finally, the Emergency-only filter shows the same rising pattern within a single admission type.

Despite our findings, what we've shown is a relationship and not a cause. More diagnoses can mean sicker patients which could drive more readmissions. Most of the encounters have exactly 9 diagnoses being the original hospital systems stored at most 9 diagnoses for most of the years. Thus patients with more conditions may have not been counted. Also, the data is from a 3000 row sample from 1999-2008 and covers only patients with diabetes meaning that the results may not apply to other hospitals or patients. In addition, some combinations are small. For example, Urgent has 74 patients with 1-4 diagnoses, so its rate is less certain.

GITHUB Website Link: https://github.com/apriljwalter-lgtm/Group_8_Midterm_project